

Day 14

Automation and Data Handling in Linux

Archiving, Compression and Cron Jobs: explained in simple language

What you will learn	Simple meaning
tar	Pack many files and folders into one file, and unpack them again
gzip, bzip2, xz	Make files smaller to save space and send faster
tar + compression	Create .tar.gz, .tar.bz2 and .tar.xz in one command
crontab	Make Linux run tasks automatically at the time you choose

Tip: Practice with test files only. Some commands here (like `rm -rf` and `crontab -r`) delete data, so read the warnings first.

1. Simple Terms You Must Know

Term	Meaning in simple words
Archive	Many files and folders packed into one file, like putting many papers in one folder. Size is not reduced.
Compression	Making a file smaller by storing its data in a cleverer way.
tarball	A common nickname for a .tar file or a compressed one like .tar.gz.
Backup	An extra copy of your data, kept safe in case the original is lost.
Cron	A background service that runs tasks automatically at set times.
Cron job	One task (command) that cron runs on a schedule.
crontab	The list (table) of your cron jobs, and also the command to manage it.
Timestamp	Date and time written as text, e.g. 20231221_153001.
Syntax	The correct way to write a command.
Option / Flag	A small extra with a dash, like -c, that changes how a command works.

Key difference: Archiving = pack together (same size). **Compression** = make smaller. They are two different jobs, and tar can do both at once.

2. Overview of Archiving

Archiving combines many files and directories into **one single file**.

Use case	Simple meaning
Backups	Keep a safe copy of your data in one file that is easy to store.
Data transfer	Send one file instead of hundreds of files.
Organization	Keep related files together and tidy.

3. Creating and Extracting Archives with tar

tar stands for **tape archive**. It is the main Linux tool for archiving.

Create an archive

Syntax: `tar -cvf archive_name.tar file1 file2 directory/`

Option	Meaning in simple words
-c	Create a new archive.
-v	Verbose: show the names of files on screen while working (progress).
-f	File: the next word is the archive file name. Always put -f last, right before the name.

```
# Example
tar -cvf project_backup.tar project_folder/
```

This packs everything inside *project_folder* into one file called *project_backup.tar*. The original folder stays as it is.

Extract an archive

Syntax: `tar -xvf archive_name.tar`

Option	Meaning
-x	Extract: unpack the files from the archive.

```
# Example
tar -xvf project_backup.tar
```

View contents without extracting

Syntax: `tar -tvf archive_name.tar` (-t = list the contents)

```
tar -tvf project_backup.tar
```

Extra: extract into a chosen folder

```
tar -xvf project_backup.tar -C /tmp/restore/
```

-C means: go to this folder first (**C**hange directory), then extract there. The folder must already exist.

Remember: Memorise the four letters: **c** = create, **x** = extract, **t** = list, **f** = file name. Always include **-f** and the archive name.

4. Introduction to Compression

Compression reduces file size by encoding the data more efficiently. It **saves storage** and makes **transfers faster**.

Tool	Compress	Decompress	Extension	Speed and size
gzip	gzip file	gunzip file.gz	.gz	Fast, good compression, works everywhere
bzip2	bzip2 file	bunzip2 file.bz2	.bz2	Slower, smaller than gzip
xz	xz file	unxz file.xz	.xz	Slowest, smallest files

gzip and gunzip

```
gzip file_name          # creates file_name.gz
gunzip file_name.gz     # gets back file_name
```

bzip2 and bunzip2

```
bzip2 file_name         # creates file_name.bz2
bunzip2 file_name.bz2   # gets back file_name
```

xz and unxz

```
xz file_name           # creates file_name.xz
unxz file_name.xz      # gets back file_name
```

Important: By default these tools **replace** the original file with the compressed one. The original disappears. To keep the original, add **-k** (keep): `gzip -k file_name`.

Remember: These three tools compress **one file** at a time. To compress a whole folder, first make a tar archive, then compress it (next section).

5. Combining Archiving and Compression

Instead of two steps, tar can archive **and** compress together by adding one extra letter.

Extra option	Compression	File name ends with
z	gzip	.tar.gz (or .tgz)
j	bzip2	.tar.bz2
J (capital)	xz	.tar.xz

Create a compressed archive

```
# gzip
tar -czvf archive_name.tar.gz file1 file2 directory/

# bzip2
tar -cjvf archive_name.tar.bz2 file1 file2 directory/

# xz
tar -cJvf archive_name.tar.xz file1 file2 directory/
```

Extract a compressed archive

```
# gzip
tar -xzvf archive_name.tar.gz

# bzip2
tar -xjvf archive_name.tar.bz2

# xz
tar -xJvf archive_name.tar.xz
```

Read -czvf letter by letter: **c** = create, **z** = use gzip, **v** = show progress, **f** = file name follows.

Extra: Modern tar can find the compression type by itself when extracting, so `tar -xvf archive_name.tar.gz` also works. Using `z`, `j` or `J` is still the classic way and good to learn.

Practical examples

Task	Command
Compress with gzip	<code>tar -czvf documents.tar.gz Documents/</code>
Compress with bzip2	<code>tar -cjvf logs.tar.bz2 Logs/</code>
Compress with xz	<code>tar -cJvf archive.tar.xz Folder/</code>
Decompress gzip	<code>tar -xzvf documents.tar.gz</code>
Decompress bzip2	<code>tar -xjvf logs.tar.bz2</code>
Decompress xz	<code>tar -xJvf archive.tar.xz</code>
List a compressed archive	<code>tar -tzvf documents.tar.gz</code>

Common mistake: Case matters: small `j` is bzip2 but capital `J` is xz. Mixing them gives an error.

6. Introduction to crontab

Cron runs tasks automatically at the time you choose, such as backups, system updates and cleanup scripts. Your schedule list is called the **crontab**.

Understanding the crontab syntax

Each cron job line has **five time fields**, then the command:

```
* * * * *    command_to_execute
| | | | |
| | | | +----- Day of the week (0 - 7) (Sunday = 0 or 7)
| | | +----- Month (1 - 12)
| | +----- Day of the month (1 - 31)
| +----- Hour (0 - 23)
+----- Minute (0 - 59)
```

Field	Allowed values	Example	Meaning
Minute	0 - 59	30	at minute 30
Hour	0 - 23 (24-hour clock)	14	at 2 PM
Day of month	1 - 31	1	on the 1st
Month	1 - 12	12	in December
Day of week	0 - 7 (0 and 7 = Sunday)	1	on Monday

Special symbols

Symbol	Meaning	Example
*	Every value (any)	* in hour = every hour
,	A list of values	1,15 = on 1 and 15
-	A range	1-5 = Monday to Friday
/	Step (every N)	* / 5 in minute = every 5 minutes

Creating and managing cron jobs

Command	What it does
crontab -l	List (view) your current cron jobs.
crontab -e	Edit your cron jobs. The first time, it asks which editor to use (nano is easy).
crontab -r	Remove: delete all your cron jobs. It does not ask for confirmation.

Warning: Be careful: `crontab -r` is one letter away from `crontab -e`. Typing it by mistake wipes your whole schedule. Tip: save a copy first with `crontab -l > mycron.txt`.

How to add a job: run `crontab -e`, type the line at the end of the file, save and exit. Cron reads it automatically.

7. Practical Cron Examples

Example 1: Backup every day at midnight

```
0 0 * * * tar -czvf /backup/home_$(date +%F).tar.gz /home/user/
```

Part	Meaning
0 0 * * *	Minute 0, hour 0 (midnight), every day, every month, every weekday.
tar -czvf ...	Create a gzip archive.
\$(date +%F)	Today's date in the form 2026-10-08, put into the file name.
/backup/	The folder must already exist.

Example 2: Clean temporary files every week

```
0 0 * * 0 rm -rf /tmp/*
```

Runs at midnight every **Sunday** (0 = Sunday). **rm** removes files, **-r** goes inside folders, **-f** forces without asking.

Danger: **rm -rf** deletes permanently with no recycle bin. A small typing mistake (like a space before **/***) can destroy the system. Test your **rm** command by hand on a practice folder first.

Example 3: System update on the 1st of every month at 2 AM

```
0 2 1 * * apt-get update && apt-get upgrade -y
```

&& means: run the second command only if the first one succeeded. **-y** means automatically answer 'yes'.

Important: Updating software needs root power. Add this line in **root's** crontab using `sudo crontab -e`. In a normal user's crontab it will fail.

Example 4: Create a file every minute

This job creates a new empty file every minute, with the date and time in its name. Files are saved in `/home/student/`.

```
* * * * * /bin/touch /home/student/minute_file_$(date +%Y%m%d_%H%M%S).txt
```

Part	Meaning
* * * * *	Run every minute.
/bin/touch	Command that creates an empty file. We write the full path because cron has a short search path.
/home/student/minute_file_...txt	Folder and file name.
\$(date +%Y%m%d_%H%M%S)	Prints the date and time as a timestamp such as 20231221_153001 . %Y = year, %m = month, %d = day, %H = hour, %M = minute, %S = second.

Very important: Inside crontab, the percent sign **%** has a special meaning (it means new line). So every **%** in a cron command must be written as **%**. On the normal command line, you write plain **%** without the backslash.

```
# Verify after one or two minutes
ls /home/student/
minute_file_20231221_153001.txt
minute_file_20231221_153101.txt
```

Clean up: Delete this job when you finish practice (run `crontab -e` and remove the line). Otherwise it creates 1,440 files every day.

More examples (extra)

Schedule	Meaning
<code>* / 5 * * * * command</code>	Every 5 minutes
<code>30 18 * * 1-5 command</code>	At 6:30 PM, Monday to Friday
<code>0 9 15 * * command</code>	At 9 AM on the 15th of every month
<code>0 0 * * * mkdir -p /home/student/daily_\$(date +%F)</code>	Create a new folder every day at midnight
<code>0 1 * * * find /backup -name "*.tar.gz" -mtime +7 -delete</code>	At 1 AM delete backup files older than 7 days
<code>@reboot command</code>	Run once when the system starts
<code>@daily command</code>	Same as <code>0 0 * * *</code>

Tip: use `mkdir -p` so there is no error if the folder already exists. The `find ... -mtime +7 -delete` command is safer than `rm -rf` because it only deletes old files of a chosen name.

8. Additional Notes: Cron Service

Cron jobs run only if the **cron service** is running.

```
sudo systemctl status cron    # check if cron is running
sudo systemctl start cron     # start it if it is stopped
sudo systemctl enable cron    # (extra) start it automatically at boot
```

Part	Meaning
systemctl	The tool that controls services (background programs).
status	Show whether the service is running.
start	Start the service now.
enable	Make it start automatically every time the computer boots.

Different systems: The service is called **cron** on Ubuntu and Debian, and **crond** on Red Hat, CentOS and Fedora (use `sudo systemctl status crond` there).

If a cron job does not work, check:

- Is the cron service running?
- Did you use **full paths** (like `/bin/touch` and `/backup/`)?
- Did you write every `%` as `%`?
- Does the folder exist and does the user have permission to write there (Day 11)?

- Does the job need root power? If yes, use `sudo crontab -e`.

9. Quick Revision Cheat Sheet

Command	What it does
<code>tar -cvf a.tar folder/</code>	Create an archive
<code>tar -xvf a.tar</code>	Extract an archive
<code>tar -tvf a.tar</code>	List archive contents
<code>tar -czvf a.tar.gz folder/</code>	Create gzip archive
<code>tar -cjvf a.tar.bz2 folder/</code>	Create bzip2 archive
<code>tar -cJvf a.tar.xz folder/</code>	Create xz archive
<code>tar -xzvf a.tar.gz</code>	Extract gzip archive (use j or J for others)
<code>gzip file / gunzip file.gz</code>	Compress / decompress one file
<code>bzip2 file / bunzip2 file.bz2</code>	Compress / decompress one file
<code>xz file / unxz file.xz</code>	Compress / decompress one file
<code>crontab -l</code>	View cron jobs
<code>crontab -e</code>	Edit cron jobs
<code>crontab -r</code>	Delete all cron jobs
<code>sudo systemctl status cron</code>	Check cron service

Compression	Speed	Size	Tar letter
gzip	Fastest	Largest of the three	z
bzip2	Medium	Smaller	j
xz	Slowest	Smallest	J

10. Practice Lab (Do in Order)

1. `mkdir -p lab/data && cd lab` (create practice folders)
2. `echo "one" > data/a.txt ; echo "two" > data/b.txt` (make test files)
3. `tar -cvf data.tar data/` (create an archive)
4. `tar -tvf data.tar` (list contents)
5. `tar -czvf data.tar.gz data/` (create compressed archive)
6. `ls -lh` (compare sizes of the .tar and .tar.gz files)
7. `mkdir restore && tar -xzvf data.tar.gz -C restore` (extract into restore/)
8. `gzip -k data/a.txt` (compress and keep the original)
9. `crontab -l` (see existing jobs)

10. `crontab -e` (add: `* * * * * /bin/touch /home/student/test_$(date +%H%M%S).txt`
)
11. `ls /home/student` (after 2 minutes, new files appear)
12. `crontab -e` (remove the test line to stop it)

Practice questions

- What is the difference between archiving and compression?
- What do the letters c, x, t, v and f mean in tar?
- Write the command to create a bzip2 compressed archive of the folder **Logs**.
- Which of gzip, bzip2 and xz gives the smallest file? Which is fastest?
- Write the cron time fields for: every Monday at 8:30 AM.
- Why must % be written as \% in a crontab line?
- What is the difference between `crontab -e` and `crontab -r`?
- How do you check whether the cron service is running?

Hint: Answer for the Monday question: `30 8 * * 1` command.

Summary: Conclusion: tar and compression tools help you store and share data in a small, tidy way. Cron lets Linux do repetitive jobs like backups, cleanup and file creation by itself, so you save time and avoid mistakes.